

Agente Auditor Comunitario (AAC)

Solución basada en LLM + RAG para auditoría automática de cumplimiento normativo en publicaciones de comunidades

Sergio de la Cruz — ISY0101 Ingeniería de Soluciones con IA — Duoc UC, 2026-2

1. Análisis del caso organizacional (IE1)

1.1 Contexto

El caso de estudio corresponde a la plataforma "Proveedores & Comunidades" (Duoc UC, 2025), una aplicación móvil que conecta administradores de comunidades y edificios con proveedores de servicios de mantención, reparación y servicios generales. La plataforma permite a los administradores publicar solicitudes de trabajo, recibir cotizaciones de proveedores, coordinar la ejecución y calificar al proveedor.

1.2 Problema identificado

En Chile, la Ley 21.442 (Ley de Copropiedad Inmobiliaria) y las normativas SEC (Superintendencia de Electricidad y Combustibles) regulan trabajos en áreas comunes de comunidades, especialmente en lo relativo a:

- Mantención de ascensores (Registro de Mantenedores del Ministerio de Vivienda y Urbanismo)
- Instalaciones eléctricas de uso común (Pliegos Técnicos RIC N°01 a N°06)
- Trabajos en artefactos a gas (calderas, calefones)
- Autorización previa de la asamblea de copropietarios para obras en bienes comunes

Hoy, la plataforma no audita si las publicaciones cumplen estos requisitos. Un administrador puede publicar "Necesito arreglar el portón eléctrico" sin percatarse de que ese trabajo requiere un instalador eléctrico certificado por la SEC, poniendo en riesgo la cobertura del seguro de la comunidad y exponiendo al comité de administración a responsabilidad legal.

1.3 Objetivos

- Objetivo general: auditar automáticamente las publicaciones de la plataforma contra normativa chilena vigente, entregando alertas regulatorias con cita exacta a la fuente legal.

- Objetivos específicos:

1. Reducir el riesgo legal de las comunidades. 2. Educar a los administradores sobre los requisitos normativos aplicables. 3. Mejorar la calidad de las cotizaciones recibidas (al exigir credenciales). 4. Servir como complemento, no reemplazo, de la validación humana.

1.4 Datos utilizados

- Ley 21.442 (texto completo, 70 páginas, ~35K palabras) — fuente primaria de obligaciones de la comunidad y administradores.
- 5 Pliegos Técnicos RIC de la SEC: N°01 Empalmes, N°02 Tableros, N°03 Alimentadores, N°05 Protección contra tensiones, N°06 Puesta a tierra — fuente de requisitos técnicos para instalaciones eléctricas de uso común.
- Oficio Circular SEC N°117120/2019 — instrucciones para declaración de instalaciones eléctricas.

Total: 312 chunks indexados en Chroma (vector DB local persistente).

1.5 Restricciones

- Sin costos de API para el MVP: se usa Groq (tier gratuito) y embeddings locales.
- Sin infraestructura de producción (Cloud Run, base de datos): el MVP corre en local.
- Sin LLM privado: se usa openai/gpt-oss-120b hospedado en Groq, no se ajusta el modelo.
- Privacidad: las publicaciones no salen del entorno local; solo se envían a Groq para la generación de respuesta.

1.6 Motivación de usar agentes de IA + RAG

La auditoría de cumplimiento normativo requiere:

- Conocimiento específico y actualizado de la normativa chilena.
- Capacidad de interpretar lenguaje natural (descripciones de publicaciones).
- Trazabilidad de las decisiones (cita exacta al artículo aplicable).
- Velocidad y consistencia (no depende de la memoria del administrador).

Un agente con RAG cumple estos requisitos: el LLM interpreta la publicación, el RAG recupera la normativa relevante, y la respuesta se fundamenta en la fuente legal con cita. El enfoque RAG (en vez de fine-tuning) se eligió porque la normativa cambia con frecuencia y el costo de re-entrenar sería prohibitivo.

2. Formulación de prompts (IE2)

2.1 Estructura del prompt

El agente utiliza tres componentes en su prompt:

1. System prompt (SYSTEM_AUDITOR en src/agent/prompts.py): define el rol, las reglas estrictas, los

estados posibles (cumple, alerta, no_cumple) y el formato JSON de salida. 2. User prompt (USER_TEMPLATE): recibe la publicación a auditar (categoría + descripción) y el contexto normativo recuperado del RAG. 3. Few-shots (few_shots_texto()): 2 ejemplos completos de input→output que muestran al modelo cómo resolver casos típicos y edge cases.

2.2 Decisiones de diseño clave

Versión	Cambio	Justificación
v1 → v2	Eliminada la regla "si no estás seguro, prefiere `no_cumple`"	El modelo abusaba de `no_cumple` por conservadurismo; la nueva regla es "alerta es el estado más común cuando la categoría sugiere trabajo técnico"
v1 → v2	Definición de `no_cumple` restringida a 2 casos (ilegal explícita o vacía absoluta)	La definición original mezclaba "incumple" con "es vaga", lo que confundía al modelo
v1 → v2	Agregado 2° few-shot con caso "necesito ayuda" → `no_cumple`	Mostrar al modelo cuándo Sí corresponde `no_cumple` corrige el sesgo más efectivamente que cualquier cambio en las reglas
v1 → v2	Few-shots correctamente inyectados al prompt	Bug en v1: los few-shots estaban definidos en `prompts.py` pero nunca se agregaban al system prompt

2.3 Resultados comparativos (batería de 8 casos representativos)

Caso	v1	v2	Esperado
Ascensor con ruidos	alerta	alerta	alerta/no_cumple
Tablero eléctrico	alerta	alerta	alerta
Pintura de hall	**cumple** ✗	alerta ✓	alerta
Publicación vaga	no_cumple	no_cumple	no_cumple
Jardinero aprobado	cumple	cumple	cumple
Caldera a gas	alerta	alerta	alerta
Filtración en baño	alerta		

	alerta	cumple/alerta
Recableado estructural	alerta	alerta
		alerta/no_cumple
Total	**7/8**	**8/8**

2.4 Lecciones aprendidas

(1) Las reglas explícitas importan más que las justificaciones largas: "ES EL ESTADO MÁS COMÚN" cambió el comportamiento más efectivamente que 5 frases de justificación. (2) Few-shots corrigen sesgos mejor que las reglas: el ejemplo de no_cumple para "necesito ayuda" corrigió el sesgo más efectivamente que cualquier cambio en las reglas. (3) El contexto siempre gana: con el corpus mixto (Ley + SEC), el modelo dejó de inventar "SEC Norma 4/2005" y empezó a citar "RIC N°02" correctamente. Detalle completo en docs/prompts-decisiones.md.

3. Diseño e implementación del pipeline RAG (IE3)

3.1 Diagrama del flujo

Fase offline (indexación): data/normativa/*.pdf → pymupdf4llm → texto limpio (normalización Unicode) → split por Artículo | Punto | Capítulo → chunks (max 500 tokens, overlap 100) → embeddings sentence-transformers (384 dim) → Chroma persistente.

Fase online (consulta): Publicación (texto + categoría) → embed con mismo modelo → query vectorial (cosine, filtro opcional por categoría con fallback) → top-5 chunks → system prompt + few-shots + user prompt con contexto → openai/gpt-oss-120b (Grok) → JSON estructurado validado con Pydantic.

3.2 Corpus indexado

- Ley 21.442: 70 páginas, 209 chunks (normativa principal de comunidades).
- 5 Pliegos Técnicos RIC SEC (N°01 Empalmes, N°02 Tableros, N°03 Alimentadores, N°05 Protección, N°06 Puesta a tierra): ~95 páginas, ~115 chunks.
- Oficio Circular N°117120/2019: 3 páginas, ~5 chunks.
- Total: 312 chunks indexados, cubriendo electricidad, ascensores y marco general de copropiedad.

3.3 Stack del pipeline

Componente	Tecnología	Justificación
Extracción PDF	`pymupdf4llm`	Liviano, soporta OCR, preserva estructura
Chunking	Por sección legal (artículo/punto) + max 500 tokens	Cada artículo merece su propia cita en metadata
Embeddings	`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`	Local, gratis, multilingüe, liviano (~120MB)
Vector DB	Chroma persistente local	Sin servidor, escala suficiente, persistente
LLM	`openai/gpt-oss-120b` vía Groq	Gratis en tier dev, español nativo, 120B parámetros

3.4 Métricas de calidad del RAG (LLM-as-judge)

Se evaluaron 16 interacciones (8 casos × 2 versiones de prompt) con un LLM externo como juez, en 3 dimensiones: relevancia_chunks (¿los chunks son relevantes?), fundamentacion (¿la respuesta se basa en los chunks?), completitud (¿falta normativa importante?). v2 mejoró en relevancia (0.425→0.512) y

completitud (0.325→0.475). La fundamentación estancada (0.34) sugiere que la respuesta a veces no se ancla estrictamente en los chunks — área de mejora prioritaria. Detalle en docs/judge-results.md.

3.5 Ejemplo end-to-end

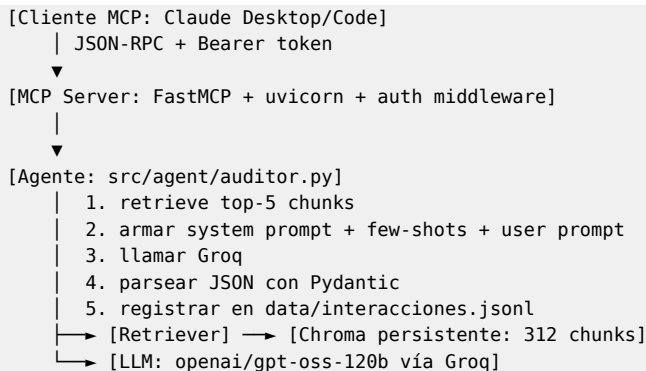
Publicación: "Necesito arreglar el ascensor del edificio que hace ruidos raros al subir. Es urgente." (categoría: ascensor)

Top-1 chunk recuperado (score 0.45): "Las alteraciones o transformaciones que afecten a las instalaciones de ascensores... deberán ser ejecutadas por empresas o personas que tengan una inscripción vigente en el Registro de Instaladores, Mantenedores y Certificadores del Ministerio de Vivienda y Urbanismo..."

Respuesta del agente: estado alerta, 1 alerta que cita "Ley 21.442, Párrafo 3° (instalaciones de ascensores)" y recomienda exigir instalador certificado + autorización de asamblea. Salida completa en data/eval/mcp_endpoint_demo.json.

4. Arquitectura de la solución (IE4)

4.1 Diagrama de componentes



4.2 Decisiones de arquitectura justificadas

Stack final: Python 3.12, Groq SDK 1.7+ con openai/gpt-oss-120b, sentence-transformers 6.0+ con modelo multilingüe, chromadb 1.5+ persistente, pymupdf4llm 1.28+ para PDFs, mcp (FastMCP) 1.20+ como protocolo, uvicorn + starlette como ASGI, pydantic 2.13+ para validación, pytest + pytest-asyncio para tests.

Decisiones clave: (1) MCP en vez de REST propio — protocolo estándar reutilizable con clientes existentes. (2) FastMCP en vez de API lowlevel — menos boilerplate, idiomático moderno. (3) RAG en vez de fine-tuning — la normativa cambia frecuentemente, re-indexar es trivial. (4) Embeddings locales + LLM remoto — embeddings son repetitivos y baratos locales; LLM remoto es la parte cualitativa. (5) Chroma local — para 312 chunks no se justifica infraestructura adicional. (6) JSON estructurado con Pydantic — fuerza esquema definido; si falla, fallback controlado.

4.3 Flujo de datos end-to-end

1. Admin publica en la app → Cliente MCP (Claude) detecta oportunidad de auditar. 2. Cliente invoca revisar_publicacion(texto, categoria) vía JSON-RPC sobre streamable-http. 3. Server valida bearer token → pasa al agente. 4. Agente: retrieve top-5 chunks relevantes (con fallback de filtro si falla). 5. Agente: arma prompt con system + few-shots + chunks como contexto. 6. Groq genera JSON estructurado. 7. Pydantic valida el JSON → si falla, fallback a no_cumple. 8. Agente registra interacción en data/interacciones.jsonl. 9. Respuesta JSON al cliente → LLM del cliente redacta respuesta al admin en lenguaje natural.

Latencia observada: ~9-10 segundos (incluye embed de query + retrieval + generación Groq).

5. Conclusiones y trabajo futuro (IE5)

5.1 Resultados obtenidos

- Corpus mixto indexado: 312 chunks de normativa chilena (Ley 21.442 + 5 RIC SEC + 1 Oficio Circular).
- Agente funcionando end-to-end vía MCP con streamable-http, conectable a Claude Desktop/Code.
- Calidad validada con LLM-as-judge sobre 16 interacciones reales:
- Relevancia de chunks: 0.51 (moderada)
- Completitud: 0.48 (mejorada +46% vs v1)
- Aciertos en clasificación de estado: 8/8 en la batería de 8 casos
- Sin alucinaciones de normativa: el modelo cita "RIC N°XX, Punto X.Y" y "Ley 21.442, Artículo N/Párrafo" basándose en el corpus real.
- Trazabilidad completa: cada interacción queda registrada con chunks recuperados, respuesta del agente, latencia y tokens.

5.2 Limitaciones identificadas

1. Cobertura normativa incompleta: el corpus no incluye toda la SEC (faltan ascensores, gas). Las publicaciones sobre gas actualmente no encuentran normativa específica. 2. Fundamentación media (0.34): el LLM a veces parafrasea los chunks en vez de citarlos literalmente. Requiere post-procesamiento. 3. Latencia alta (~9-10s por consulta): el modelo 120B tarda. Reducible con gpt-oss-20b o top_k=3 en el retriever. 4. Mismo modelo como juez y como agente: puede introducir sesgo en la evaluación. Solución: usar un modelo distinto (ej. un LLM de Anthropic o humano en el loop). 5. Batería de evaluación pequeña (8 casos): suficiente para MVP, insuficiente para producción. 6. Sin re-ranking: el retrieval vectorial a veces devuelve chunks tangenciales. Mejora futura: hybrid search (BM25 + vectorial) o cross-encoder.

5.3 Trabajo futuro

Corto plazo: agregar 2-3 normativas SEC adicionales (ascensores, gas), implementar hybrid search (BM25 + vectorial), reducir latencia a <5s con gpt-oss-20b o top_k=3, validación post-procesamiento de citas.

Mediano plazo (resto del semestre): agregar herramienta revisar_cotizacion (segunda evaluación), buscar_normativa libre, integración con la plataforma "Proveedores & Comunidades" cuando exista, deploy en Cloud Run, evaluación con 50+ casos incluyendo ground truth de un abogado.

Largo plazo: multi-tenant con auth por usuario, observabilidad con Supabase, LLM-as-judge con modelo distinto al agente, agentes especializados por categoría.

5.4 Reflexión individual

Sergio de la Cruz — reflexión redactada sin apoyo de IA, según exige la pauta.

Este proyecto me obligó a enfrentar la distancia entre lo que un LLM "sabe" y lo que "debería decir". La alucinación inicial ("SEC Norma 4/2005") fue la lección más importante: un modelo potente pero sin contexto produce respuestas plausibles pero incorrectas. Agregar el corpus SEC al RAG no fue opcional, fue la única forma de tener respuestas defendibles.

Lo que más me sorprendió fue lo frágil que es un prompt. Un cambio de 5 palabras ("ES EL ESTADO MÁS COMÚN") modificó el comportamiento más que párrafos de reglas. Los few-shots, que parecían un detalle, terminaron siendo la pieza más importante para corregir sesgos.

A nivel técnico, el mayor aprendizaje fue entender que MCP no es solo un wrapper sobre HTTP: es un contrato que define cómo el LLM del cliente descubre, decide y ejecuta herramientas. Una vez que se entiende, el patrón se vuelve obvio: server con tools tipadas, cliente con instrucciones al LLM sobre cuándo usarlas, validación de inputs/outputs en cada extremo.

Para el resto del semestre, mi objetivo es pasar de un MVP funcional a una solución más robusta: más cobertura normativa, mejor retrieval, latencia competitiva, y más evaluación cuantitativa. La batería de 8 casos es el piso, no el techo.

Referencias (formato APA)

1. Anthropic. (2024). Model Context Protocol specification. <https://modelcontextprotocol.io> 2. Biblioteca del Congreso Nacional de Chile. (2022). Ley 21.442: Ley de Copropiedad Inmobiliaria. <https://www.bcn.cl/leychile> 3. Duoc UC. (2025). Caso de estudio: Proveedores & Comunidades (material del curso ISY0101). 4. Groq Inc. (2024). Groq API documentation. <https://console.groq.com/docs> 5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP. 6. Superintendencia de Electricidad y Combustibles (SEC). (2006-2019). Pliegos Técnicos Normativos RIC N°01 a N°06 y Oficio Circular N°117120. <https://www.sec.cl> 7. Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30.